
GPU Kernel Design Patterns

A deep-dive into `cov_reduce_r3_sym6_warp`

Written for a physicist who programmes — not a computer scientist

Before We Begin: How a GPU Actually Works

Before examining any single technique, it helps to have a concrete mental model of GPU hardware. Skipping this step is the reason most GPU code feels like magic to newcomers.

The GPU is a city of tiny, specialised workers

A modern GPU — say, an NVIDIA A100 — contains thousands of **CUDA cores**. Each core is far simpler and slower than a CPU core; it cannot do branch prediction, out-of-order execution, or most of the tricks that make a CPU fast for serial code. What it *can* do is execute simple arithmetic operations in parallel across enormous datasets. The GPU wins by sheer weight of numbers, not individual cleverness. When you write a CUDA kernel, you write the instructions that a single thread will execute, and then launch millions of copies of that thread simultaneously.

The thread, warp, and block hierarchy

GPU threads are organised in a strict three-level hierarchy that you must always keep in mind:

- **Thread** — the smallest unit of work. It has its own registers (fast, private scratch space) and executes one stream of instructions. Think of it as a single worker with a notepad.
- **Warp** — a fixed group of **32 threads** that the hardware always schedules and executes *together*, in lock-step, running the same instruction at the same moment. This is called **SIMT** (Single Instruction, Multiple Threads). You do not choose the warp size; it is always 32 on every NVIDIA GPU ever made. Think of a warp as a choir that must all sing the same note at the same instant.
- **Block** — a group of warps (and therefore threads) that share a region of fast on-chip memory called **shared memory**. Threads in the same block can communicate cheaply via shared memory. Threads in different blocks cannot communicate directly at all.
- **Grid** — the collection of all blocks launched by a single kernel call. In this kernel, each block in the grid handles one baseline group.

The memory hierarchy and why it matters enormously

On a GPU, not all memory is equal. Access latency differs by orders of magnitude:

Memory type	Latency (approx.)	Scope
Registers	~1 cycle	Per-thread (private)
Shared memory	~10 cycles	Per-block (shared among all threads in block)
L1/L2 cache	~30–100 cycles	Device-wide (managed automatically)
Global (VRAM)	~400–800 cycles	Device-wide (explicit programmer control)

The central challenge of GPU programming is keeping your data in fast memory (registers and shared memory) and minimising expensive round-trips to global VRAM. Every technique in this kernel is — directly or indirectly — an attempt to do exactly that.

1. What Does This Kernel Actually Compute?

The physical problem is a weighted outer-product sum over antenna baselines. For each baseline group b , the kernel accumulates the 3×3 symmetric matrix:

$$C_b = \sum_{i \in \mathcal{S}_b} \frac{1}{w_i} \mathbf{d}_i \mathbf{d}_i^\top \quad (1)$$

where $\mathbf{d}_i \in \mathbb{R}^3$ is a residual vector (stored row-wise in the array $\mathbf{D}$) and w_i is a noise weight stored in $\mathbf{w}$. This structure is identical to the normal equations matrix in weighted least-squares, or the Fisher information matrix for a Gaussian likelihood with per-sample heteroscedastic noise — both extremely familiar objects in observational physics.

The kernel parallelises this computation across all baseline groups simultaneously, with multiple threads within a block collaborating on each group to sum over potentially millions of data points per group.

2. Technique 1: Warp Shuffle Reduction

The problem it solves

Suppose you have 32 numbers — one held by each thread in a warp — and you want their sum. The naive approach would be to write each thread's value to shared memory, synchronise all threads, then have one thread loop over all 32 values. This costs a shared-memory write, a synchronisation barrier, and 32 serial additions.

A much faster approach exploits the fact that threads within a warp execute in lock-step: they can exchange register values directly with each other using **warp shuffle instructions**, bypassing shared memory entirely.

How it works: the parallel butterfly reduction

The `warp_sum` function implements a *parallel reduction* using the `__shfl_down_sync` hardware intrinsic:

```

1  __device__ __forceinline__ float warp_sum(float v) {
2      unsigned mask = 0xffffffffu; // all 32 bits set: all lanes
   participate
3      v += __shfl_down_sync(mask, v, 16);
4      v += __shfl_down_sync(mask, v, 8);
5      v += __shfl_down_sync(mask, v, 4);
6      v += __shfl_down_sync(mask, v, 2);
7      v += __shfl_down_sync(mask, v, 1);
8      return v;
9  }
```

Listing 1. `warp_sum`: reduces a float across all 32 lanes in a warp

Each call `__shfl_down_sync(mask, v, delta)` causes every thread to receive the value of `v` held by the thread `delta` positions *ahead* of it in the warp. Here is a worked example with 8 threads for visual clarity (the real warp has 32):

Lane	0	1	2	3	4	5	6	7
Initial value	a	b	c	d	e	f	g	h
After <code>delta=4</code>	$a+e$	$b+f$	$c+g$	$d+h$	e	f	g	h
After <code>delta=2</code>	$a+c+e+g$	$b+d+f+h$	$c+g$	$d+h$	...			
After <code>delta=1</code>	$a+b+\dots+h$	$b+\dots$	...					

After $\log_2(32) = 5$ steps, lane 0 holds the sum of all 32 values. The full-mask `0xffffffff` (32 bits, all set to 1) tells the hardware that all 32 lanes are valid participants.

`__shfl_down_sync` is a *single* hardware instruction that reads a value from another thread's register file directly. No shared memory allocation, no `__syncthreads()` barrier, no cache traffic. The five shuffle calls together are faster than a single shared-memory write-read round-trip.

Think of this like a round-robin tournament for summing. In round 1, thread 0 adds the value from thread 16, thread 1 from thread 17, and so on — all 32 additions happen simultaneously. In round 2, thread 0 adds from thread 8. After 5 rounds, thread 0 holds the total. This is the same $\mathcal{O}(\log N)$ parallel reduction that underpins the FFT algorithm; the structure is identical.

3. Technique 2: Two-Level Block Reduction

Scaling beyond one warp

A warp is 32 threads, but a CUDA block can contain up to 1024 threads. This kernel is launched with `threads=128`, giving $128 / 32 = 4$ warps per block. After each warp finishes its own `warp_sum`, we have 4 partial sums — one per warp — but not yet a single final answer. We need a second reduction pass across warps.

The kernel handles this with a careful three-stage protocol:

Stage 1 — each warp reduces its 32 threads:

```

1 // Each thread has accumulated its own partial sum in acc[0..5].
2 // warp_sum reduces those across the 32 lanes; lane 0 of each
3 // warp ends up holding that warp's total contribution.
4 for (int t = 0; t < 6; ++t) acc[t] = warp_sum(acc[t]);

```

Listing 2. Stage 1: every warp independently reduces its local partial sums

Stage 2 — lane 0 of each warp writes its result to shared memory:

```

1 int lane = (int)(threadIdx.x & 31); // lane index within warp (0 to 31)
2 int warp = (int)(threadIdx.x >> 5); // warp index within block (0 to 3)
3 int num_warps = (int)(blockDim.x >> 5);
4
5 extern __shared__ float sh[];

```

```

6
7  if (lane == 0) {
8      for (int t = 0; t < 6; ++t)
9          sh[t * num_warps + warp] = acc[t]; // each warp writes one column
10 }
11 __syncthreads(); // all warps must finish writing before anyone reads

```

Listing 3. Stage 2: warp totals deposited into shared memory for cross-warp communication

Stage 3 — warp 0 reads all partial sums and reduces again:

```

1  if (warp == 0) {
2      float v[6];
3      // Load the per-warp partial sums into lanes 0..num_warps-1 of warp 0.
4      // Lanes beyond num_warps load zero (they will vanish in the sum).
5      if (lane < num_warps) {
6          for (int t = 0; t < 6; ++t) v[t] = sh[t * num_warps + lane];
7      } else {
8          for (int t = 0; t < 6; ++t) v[t] = 0.0f;
9      }
10     // One more warp reduction: lane 0 now holds the block's final answer.
11     for (int t = 0; t < 6; ++t) v[t] = warp_sum(v[t]);
12 }

```

Listing 4. Stage 3: warp 0 gathers partial sums from shared memory and performs the final reduction

We could write every thread’s value to shared memory and run a conventional tree reduction. That would require 128 shared-memory writes and multiple synchronisation barriers. Instead, *only lane 0 of each warp touches shared memory* — just 4 writes instead of 128. Most of the work occurs in the fast warp-shuffle path; the slow shared-memory path is used only for the final inter-warp hand-off.

Understanding the lane and warp index arithmetic

The two bit-manipulation expressions are worth unpacking:

- `threadIdx.x & 31` — bitwise AND with 31 (0b00011111) extracts the bottom 5 bits of the thread index, giving the lane number within the current warp (0–31). This is mathematically equivalent to `threadIdx.x % 32` but is a single bitwise operation.
- `threadIdx.x » 5` — a right-shift by 5 (i.e. division by 32) gives the warp index within the block (0, 1, 2, or 3 for a 128-thread block). Equivalent to `threadIdx.x / 32`.

4. Technique 3: Dynamic Shared Memory

What is shared memory?

Shared memory is a small (typically 48–164 KB depending on GPU model) but fast on-chip SRAM pool allocated *per block*. Every thread within a block can read and write to it. It is the GPU’s equivalent of a programmer-controlled L1 cache: fast like a cache, but you decide what goes in it and when.

In this kernel, shared memory serves purely as a *message-passing relay* between warps. It

holds the per-warp partial sums temporarily while warp 0 collects and reduces them.

Dynamic allocation: size set at launch time

Shared memory can be allocated statically (size baked into the kernel source at compile time) or *dynamically* (size supplied by the caller at launch time). This kernel uses the dynamic form:

```
1 extern __shared__ float sh[]; // no size given here -- set at launch
```

Listing 5. Dynamic shared memory: the array size is unspecified in the kernel

The required size is computed in Python and passed when the kernel is invoked:

```
1 num_warps = threads // 32
2 # 6 matrix entries, one float per warp per entry
3 shared_bytes = 6 * num_warps * 4 # 4 bytes per float32
4 k_r3((nb,), (threads,), (D, w, edges, out, np.int32(nb)), shared_mem=
    shared_bytes)
```

Listing 6. Shared memory size computed and injected from the Python side

The layout within the array is a $6 \times \text{num_warps}$ matrix in row-major order, where each row corresponds to one of the 6 symmetric matrix entries, and each column corresponds to one warp:

$$\text{sh}[\underbrace{t}_{\text{matrix entry index}} \times \text{num_warps} + \underbrace{\text{warp}}_{\text{warp index}}]$$

Dynamic allocation has no runtime cost; the memory is reserved on the GPU at launch time, before the kernel begins executing.

After the warps write to shared memory, `__syncthreads()` must be called before warp 0 reads. Without this barrier, warp 0 could read stale or uninitialised values. Unlike CPUs, warps within a block are *not* guaranteed to execute in any particular order relative to each other — barriers are the only way to enforce ordering.

5. Technique 4: Exploiting Symmetric Matrix Structure

The mathematical observation

The matrix C_b defined in Equation (??) is symmetric by construction: $(C_b)_{jk} = \sum_i w_i^{-1} d_{ij} d_{ik} = (C_b)_{kj}$. A 3×3 symmetric matrix has only $3 + 2 + 1 = 6$ unique entries (the diagonal plus the upper triangle). Computing all 9 entries independently would waste approximately one third of the arithmetic work, repeated in every iteration of the inner loop over potentially millions of data points.

The triangular storage convention

The kernel accumulates only 6 scalars in the register array `acc[0..5]`, indexed by the following convention:

$$\begin{aligned}
 \text{acc}[0] &= \sum_i w_i^{-1} d_{i,0}^2 & (0,0) \\
 \text{acc}[1] &= \sum_i w_i^{-1} d_{i,0} d_{i,1} & (0,1) \\
 \text{acc}[2] &= \sum_i w_i^{-1} d_{i,1}^2 & (1,1) \\
 \text{acc}[3] &= \sum_i w_i^{-1} d_{i,0} d_{i,2} & (0,2) \\
 \text{acc}[4] &= \sum_i w_i^{-1} d_{i,1} d_{i,2} & (1,2) \\
 \text{acc}[5] &= \sum_i w_i^{-1} d_{i,2}^2 & (2,2)
 \end{aligned}$$

```

1  float d0 = D[i*3 + 0];
2  float d1 = D[i*3 + 1];
3  float d2 = D[i*3 + 2];
4
5  // Compute the weight factor once, then multiply into d0, d1, d2.
6  // Each wd_k is reused up to 3 times -- no redundant division.
7  float wd0 = wi * d0;
8  float wd1 = wi * d1;
9  float wd2 = wi * d2;
10
11 acc[0] += wd0 * d0;    // (0,0)
12 acc[1] += wd0 * d1;    // (0,1)
13 acc[2] += wd1 * d1;    // (1,1)
14 acc[3] += wd0 * d2;    // (0,2)
15 acc[4] += wd1 * d2;    // (1,2)
16 acc[5] += wd2 * d2;    // (2,2)

```

Listing 7. Upper-triangle accumulation: 6 products instead of 9

The pre-multiplication into `wd0`, `wd1`, `wd2` means w_i^{-1} is applied exactly 3 times (once per component of $\mathbf{d}_i$) rather than 6 times (once per matrix entry). This is a small but free saving on top of the triangular reduction.

Reconstruction at write time

At the end, the 6 accumulated values are written into the full 3×3 output by simply mirroring the off-diagonal entries:

```

1  // v[] ordering: [00, 01, 11, 02, 12, 22]
2  // Written into a 3x3 row-major block (9 floats):
3  out[0] = v[0]; out[1] = v[1]; out[2] = v[3];
4  out[3] = v[1]; out[4] = v[2]; out[5] = v[4];
5  out[6] = v[3]; out[7] = v[4]; out[8] = v[5];

```

Listing 8. Reconstructing the full 3×3 matrix from 6 unique values at write time

This write happens once per block, so its cost is entirely negligible. The savings from the

triangular computation occur on every data point in the inner loop, which is where the overwhelming majority of time is spent.

6. Technique 5: Batched Segmented Reduction via edges

The physical motivation

In a radio telescope array, data is naturally partitioned by baseline (each pair of antennas). The covariance matrix must be computed independently for each baseline group, and these groups may have different sizes (different numbers of time and frequency samples per baseline). This is an *irregular batched* computation: many independent reductions, each over a segment of potentially different length.

The CSR encoding

The `edges` array encodes segment boundaries using the **Compressed Sparse Row (CSR)** pointer convention, which is the same format used to store sparse matrices in SciPy's `csr_matrix`. The array has `nb+1` entries; the b -th segment spans rows `edges[b]` to `edges[b+1]-1` inclusive:

Index:	0	1	2	3	...		<code>nb</code>
<code>edges</code> :	0	47	91	140	...		N
Segment 0 spans rows 0–46 Segment 1 spans rows 47–90 etc.							

Each CUDA block dispatches to one segment:

```

1  int b      = (int)blockIdx.x;  // which segment (baseline group) this block
        owns
2  int start = edges[b];          // index of first row in this segment
3  int stop  = edges[b+1];        // one past the last row (standard half-open
        convention)

```

Listing 9. Each block reads its own segment boundaries from the `edges` array

All `nb` blocks execute concurrently. Blocks assigned to large segments simply run their inner loop more times; blocks with small segments finish early and free their hardware resources. Because blocks never communicate, there is zero synchronisation overhead between them.

Think of it like distributing a pile of particle collision events among independent histogram-filling processes. Each process owns its own histogram (the 3×3 output matrix), is handed its own list of events (the segment), and fills without needing to coordinate with the others. The CSR array is the dispatch table that tells each process where to find its events.

7. Technique 6: Coalesced Global Memory Access

Why memory access patterns matter so much

Global memory (VRAM) on a GPU is served by a memory controller that fetches data in aligned **128-byte cache lines**. If 32 threads (a full warp) simultaneously request addresses

that all fall within the same cache line, the hardware satisfies all 32 requests in a *single* memory transaction. If they instead request 32 scattered, non-contiguous addresses, the hardware may need up to 32 separate transactions — a $32\times$ slowdown in effective memory bandwidth.

The pattern of having a warp’s threads access consecutive memory addresses is called **memory coalescing**, and it is frequently the single most impactful optimisation in GPU code, especially for memory-bandwidth-limited kernels like this one.

How the kernel achieves coalescing

The inner accumulation loop iterates with a stride equal to the full block size T :

```

1  int tid = (int)threadIdx.x;    // this thread's index within the block
   (0..127)
2  int T    = (int)blockDim.x;    // total threads in block (128)
3
4  for (int i = start + tid; i < stop; i += T) {
5      float wi = 1 / w[i];        // w is laid out as w[0], w[1], w[2], ...
6      float d0 = D[i*3 + 0];      // D row i, column 0
7      float d1 = D[i*3 + 1];      // D row i, column 1
8      float d2 = D[i*3 + 2];      // D row i, column 2
9      ...
10 }
```

Listing 10. Strided loop: consecutive threads access consecutive memory addresses

On the first iteration of the outer loop, thread 0 accesses row `start+0`, thread 1 accesses row `start+1`, ..., thread 31 accesses row `start+31`. These are 32 consecutive entries in memory — exactly one 128-byte cache line per float32 array element. The memory controller satisfies the entire warp’s load request in a single transaction. On the next iteration (offset by $T=128$), the same coalescing property holds.

If instead every thread iterated over all rows in the segment sequentially (the natural CPU-style loop), then at any given moment all 32 threads in a warp would be requesting the *same* address (they are executing the same instruction on the same iteration). 31 of the 32 requests would be redundant and serialised. This is the worst possible access pattern on a GPU, and is the most common mistake made by programmers porting serial code.

8. Technique 7: Compiler Loop Unrolling and Forced Inlining

What `#pragma unroll` does

Several loops in this kernel iterate over a fixed, compile-time-known number of elements (6, for the symmetric matrix entries). The `#pragma unroll` directive instructs the compiler to *unroll* these loops: replace the loop with 6 consecutive copies of the loop body, eliminating the branch instruction (“have we finished the loop yet?”) and the counter-increment instruction from every iteration.

```

1  #pragma unroll
2  for (int t = 0; t < 6; ++t) acc[t] = warp_sum(acc[t]);
```

```

3
4 // The compiler emits the equivalent of:
5 // acc[0] = warp_sum(acc[0]);
6 // acc[1] = warp_sum(acc[1]);
7 // acc[2] = warp_sum(acc[2]);
8 // acc[3] = warp_sum(acc[3]);
9 // acc[4] = warp_sum(acc[4]);
10 // acc[5] = warp_sum(acc[5]);

```

Listing 11. `#pragma unroll` on the 6-entry reduction loop

The result is a straight-line sequence of 30 shuffle instructions (6×5 steps) with no conditional branches anywhere in between. GPU hardware executes branches poorly (branches force divergence within a warp), so eliminating them is always desirable when the trip count is small and fixed.

Why unrolling also helps the register allocator

After unrolling, the compiler sees all six `acc[t]` values simultaneously. It can assign each to a separate register, making their availability explicit to the instruction scheduler. In a rolled loop, the compiler may be uncertain whether `acc[t]` from one iteration aliases or overlaps with values from another (especially in more complex contexts), limiting its optimisation freedom. With unrolling, all uncertainty is removed.

`__forceinline__` on `warp_sum`

The `__forceinline__` attribute on `warp_sum` instructs the compiler to always substitute the function body directly at each call site, rather than generating an actual function call.

A function call on a GPU is expensive: the hardware must save the current register state (pushing to a “local memory” stack, which resides in slow global VRAM), jump to the function, execute it, then restore the state and return. By inlining, all five shuffle instructions appear *directly* in the calling kernel’s instruction stream. The compiler can then intermix them with surrounding arithmetic, potentially hiding memory latency by overlapping independent operations.

For a loop body this simple — a few multiply-adds and five shuffles — the per-iteration overhead of a branch and a counter increment is actually a non-trivial fraction of total instruction count. Unrolling removes a measurable fraction of the GPU’s instruction-issue burden. For more complex loop bodies, the relative benefit would be smaller.

9. Summary of All Techniques

Technique	Location in code		What bottleneck it addresses
Warp shuffle reduction	<code>warp_sum</code>		Synchronisation latency: sums 32 values in 5 register-level instructions with no shared-memory traffic
Two-level block reduction	<code>cov_reduce_r3</code>	main body	Scalability: extends the fast warp reduction to blocks containing multiple warps
Dynamic memory	<code>extern __shared__ float sh[]</code>		Inter-warp communication: provides on-chip temporary storage; size adapted at launch
Symmetric structure	<code>accumulate_sym6_r3</code>		Arithmetic throughput: reduces multiply-add count by $\approx 33\%$ per data point
Batched segmented reduction	<code>edges</code> array	<code>+ blockIdx.x</code>	Parallelism across the problem: all baseline groups processed simultaneously
Coalesced memory	Strided loop	in <code>accumulate</code>	Memory bandwidth: ensures each warp's loads are satisfied in a single transaction
Loop unrolling + inlining	<code>#pragma unroll,</code> <code>__forceinline__</code>		Instruction overhead: eliminates branches, improves register allocation and scheduling

Each technique attacks a different bottleneck. *Coalescing* addresses memory bandwidth. *Warp shuffles* address synchronisation latency. *Symmetric exploitation* addresses arithmetic throughput. *Segmented reduction* addresses parallelism at the problem level. *Loop unrolling* addresses instruction-issue overhead. A well-tuned GPU kernel must attack all of these simultaneously. This is what distinguishes a kernel that runs at 10% of peak hardware throughput from one that runs at 70%: not a single clever trick, but the careful, layered elimination of every significant inefficiency.